1. Water Jug Problem (4-gallon and 3-gallon jugs to get 2 gallons)

- **Initial State:** $(0,0)$ where the first coordinate is the amount in the 4-gallon jug and the second is in the 3-gallon jug.
- **Goal State:** $(2, y)$ where y can be anything (we want exactly 2 gallons in the 4-gallon jug).

Step-by-Step Solution Sequence:

1. **Fill the 3-gallon jug:**
    - State: $(0,3)$
2. **Pour all water from the 3-gallon jug into the 4-gallon jug:**
    - State: $(3,0)$
3. **Fill the 3-gallon jug again:**
    - State: $(3,3)$
4. **Pour water from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full:**
    - Since the 4-gallon jug already has 3 gallons, it only takes 1 gallon to fill it completely. This leaves $3 - 1 = 2$ gallons in the 3-gallon jug.
    - State: $(4,2)$
5. **Empty the 4-gallon jug onto the ground:**
    - State: $(0,2)$
6. **Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug:**
    - State: $(2,0)$

- **Final Result:** Exactly 2 gallons of water are now measured in the 4-gallon jug.

2. Mars Rover Analysis

- **i. What are the percepts for this agent?**
    - Camera images of the terrain and rocks, spectrometer readings of soil and rock samples, atmospheric pressure/temperature data, wheel touch/slip sensors, and obstacle proximity detection.
- **ii. Characterize the operating environment:**
    - **Partially Observable:** The rover only senses its immediate surroundings and targeted sample sites.

    - **Deterministic / Stochastic:** Partly deterministic in movement, but stochastic due to unpredictable terrain texture, dust storms, and wheel slippage.
    - **Sequential:** Current actions (e.g., drilling or path selection) directly affect future states and battery levels.
    - **Static / Semi-dynamic:** The Martian environment changes slowly, but surface obstacles or wheel-slip conditions change dynamically.
    - **Discrete / Continuous:** Continuous for physical movement and coordinates, but discrete for specific decision states (e.g., move forward, drill, analyze).
- **iii. What are the actions the agent can take?**
    - Drive (forward, backward, turn left, turn right), deploy robotic arm, drill into rocks, collect samples, run onboard spectrometer analysis, and transmit data back to Earth.
- **iv. How can one evaluate the performance of the agent?**
    - Quantity and quality of valid geological samples collected, accuracy of sample analysis, successful safe distance traveled, energy/battery efficiency, and timeliness of data transmission.
- **v. What sort of agent architecture do you think is most suitable for this agent? Why?**
    - **Model-Based Reflex Agent with State and Goal-Based capabilities.** Why: The rover operates in an environment with time delays in communication from Earth, requiring autonomy. It needs an internal model of its surroundings, battery levels, and mission goals to make rational decisions when unexpected obstacles or interesting geological formations are encountered.

3. 8-Queens Problem Formulation

- **Initial State:** An empty chessboard with zero queens placed (0×0 board configuration or empty columns).
- **States:** Any arrangement of 0 to 8 queens on the board.
- **Actions:** Add a queen to any empty square in the next available column.
- **Transition Model:** Returns the board with a new queen added to the specified row and column.
- **Goal Test:** 8 queens are on the board such that no two queens threaten each other (i.e., no two queens share the same row, column, or diagonal).
- **Path Cost:** Cost of 1 per queen placed (or total steps to reach the goal configuration where cost equals 8 placements).

4. OLA Cab Booking Problem-Solving Agent & Pseudocode

- Type of Problem-Solving Agent: Goal-Based Agent (or Utility-Based Agent, since it evaluates options like mini, micro, sedan, shared, and prime based on comfort, price, and ETA to maximize utility).

Pseudocode:

```
function OLA_Cab_Booking_Agent(pickup_location, destination, preferences):
    # Goal: Transport customer from pickup to destination safely and comfortably

    available_cabs = Get_Nearby_Cabs(pickup_location)
    filtered_cabs = []

    for each cab in available_cabs:
        if cab.type matches preferences.preferred_types:
            estimated_fare = Calculate_Fare(cab, pickup_location, destination)
            eta = Calculate_ETA(cab, pickup_location)
            utility_score = Evaluate_Utility(cab, estimated_fare, eta, preferences)
            filtered_cabs.append((cab, utility_score))

    if filtered_cabs is empty:
        return "No cabs available matching preferences"

    # Select the cab that maximizes the user's utility (comfort vs cost vs time)
    best_option = Select_Max_Utility(filtered_cabs)

    booking_status = Confirm_Booking(best_option.cab, pickup_location, destination)

    return booking_status
```

5. Least-Cost Path using Uniform Cost Search (UCS)

Given the delivery network graph:

PDF

- **Start:** S
- **Goal:** G

Step-by-Step UCS Expansion:

1. **Initialize Priority Queue:** [(S, cost = 0)]
2. **Expand S:**
    - Path S → A(Cost: 1)
    - Path S → G(Cost: 12)
    - Queue: [(A, 1), (G, 12)]
3. **Expand A(cost 1):**
    - Path S → A → B(Cost: $1 + 3 = 4$)
    - Path S → A → C(Cost: $1 + 1 = 2$)
    - Queue: [(C, 2), (B, 4), (G, 12)]
4. **Expand C(cost 2):**
    - Path S → A → C → D(Cost: $2 + 1 = 3$)
    - Path S → A → C → G(Cost: $2 + 2 = 4$)
    - Queue: [(D, 3), (B, 4), (G, 4 - via C), (G, 12 - direct)]
5. **Expand D(cost 3):**
    - Path S → A → C → D → G(Cost: $3 + 3 = 6$)
    - Queue: [(B, 4), (G, 4 - via C), (G, 6), (G, 12)]
6. **Next lowest cost node is B(cost 4) or G(via C, cost 4):**
    - Reaching Gvia path S → A → C → Ggives a total cost of **4**.

Final Result:

- **Optimal Path:** S → A → C → G
- **Total Minimum Cost: 4**